

Data Science Collective

This member-only story is on us. [Upgrade](#) to access all of Medium.

★ Member-only story

A Qwen 3.5 122B LLM on a 16 GB Mac mini: MoE Expert Streaming with TurboQuant-MLX



Manjunath Janardhan

Follow

13 min read · May 25, 2026



1.3K



24



19



Per-expert disk streaming runs a 122-billion-parameter Mixture-of-Experts model — 3× bigger than RAM — on the cheapest Mac Apple sells. Bit-identical output, no swap, no sysctl tweaks.



Image by Manjunath Janardhan. A 122B-parameter LLM running on a 16 GB Mac mini via TurboQuant-MLX expert streaming]

I ran a **122-billion-parameter** language model on a **\$599 Mac mini** with **16 GB of RAM**. No, that's not a typo — and the output is coherent.

The model is **Qwen3.5-122B-A10B**, a 256-expert Mixture-of-Experts. In BF16 it's ~240 GB. Quantized to 3-bit with **TurboQuant-MLX** it's still ~54 GB on disk — more than **3× the machine's entire RAM**. It has no business running on a 16 GB Mac. It runs anyway, because of a technique called **expert streaming**: instead of loading all 256 experts into memory, it pages only the ~8 each token actually uses straight off the SSD, behind a small cache. The model on disk is 54 GB; the resident footprint peaks at ~9 GB.

I didn't start at 122B. I started with a 35B model to prove the idea, watched it fit **under 4 GB**, and then ran the same code on a 122B to see how far it would go. This article is both: the technique, validated on a 35B, then pushed to a 122B — and the surprising lesson that fell out of it, which is that **for a sparse MoE, the “memory wall” is really a disk-bandwidth wall**.

This is Part 5 of a series on TurboQuant for Apple Silicon. In [Part 1][p1] I adapted Google’s TurboQuant for dense-model weight compression on MLX. In [Part 2][p2] I extended it to Mixture-of-Experts (GPT-OSS-120B at 44 tok/s, Qwen3.5–122B at 26.5 tok/s on a 64 GB Mac). In [Part 3][p3] I added KV cache compression. In [Part 4][p4] I designed per-path hybrid quantization to fit a 120B on a 48 GB MacBook. Every part traded *quantization* for *fit*. This one trades *disk bandwidth* for *fit*, and it lands on 16 GB — the tier most people actually own.

[p1]:

I Built a Quantization Method That Beats Standard 4-bit on a 7B Model — With Zero Training Data Adapting Google’s TurboQuant for weight compression on Apple Silicon using MLX, and what the perplexity numbers... ai.gopubby.com	
---	--

[p2]:

How I run 122B-parameter LLMs on a MacBook — outperforming MXFP4 and standard quantization on... I extended TurboQuant to Mixture-of-Experts models. A 120B LLM that no existing format could fit on 64GB now runs at 44... medium.com	
--	--

[p3]:

TurboQuant : Compressing KV cache 4x on Apple Silicon — how I doubled the usable context length. medium.com	
--	--

[p4]:

Nemotron 120B on a 48 GB MacBook: 27 tok/s with TurboQuant Hybrid Quantization (MLX)

Per-path hybrid 3-bit/2-bit quantization fits a 120B Mamba + MoE model in 36 GB on disk and 40.8 GB peak — runs on...

medium.com

Want to try it as you read? On any Apple Silicon Mac (16 GB is enough):

```
pip install "turboquant-mlx-full>=0.4.1"

python -m turboquant_mlx.stream.stream_generate \
  model manjunathshiva/qwen3.5-122b-tq3 \
  prompt "Explain why the sky is blue." \
  max-tokens 128 - cache-budget-gb 4
```

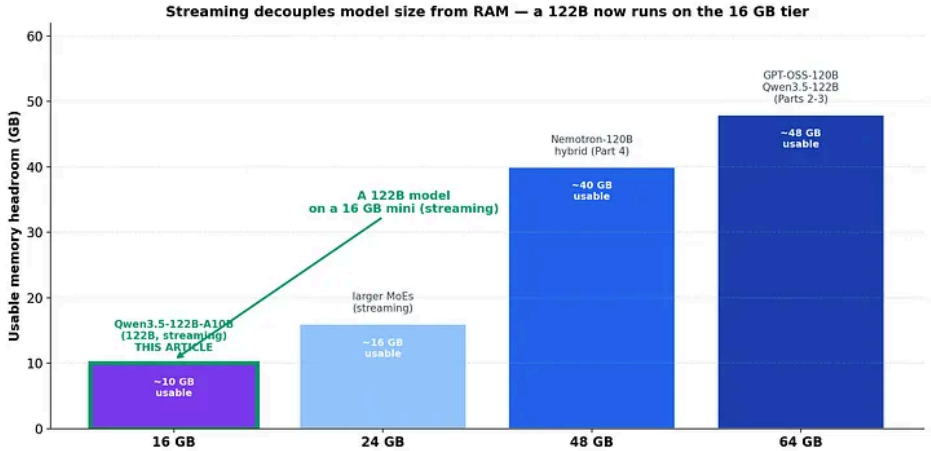
A complete copy-pasteable recipe is at the end in [Try it yourself]

Why 16 GB is the tier that matters

Parts 2–4 of this series each celebrated a smaller machine: 64 GB, then 48 GB. But the truth about Apple Silicon is that the overwhelming majority of machines in the wild are **16 GB** — the base MacBook Air, the 14" MacBook Pro, the Mac mini. If a technique needs 48 GB, it doesn't run on the laptop in most people's bags or the mini on most people's desks.

And 16 GB is *brutal* for large models. After the OS and your apps, you have maybe 10–12 GB of usable headroom. Quantization alone doesn't save you: even at an aggressive 3-bit, a 35B model is ~16 GB on disk and peaks ~18 GB when fully resident — over the line before you open a browser tab. A 122B at 3-bit is ~54 GB. The resident is hopeless.

RAM VS. MODEL SIZE ON APPLE SILICON: THE POWER OF EXPERT STREAMING. BENCHMARK COMPARISON OF LARGEST LOADABLE MODELS.		
APPLE SILICON TIER	LARGEST *FULLY-RESIDENT* TURBOQUANT MODEL	WITH EXPERT STREAMING (THIS ARTICLE)
16 GB	GPT-OSS-20B (3-bit, 9.3 GB)	SPECIAL BREAKTHROUGH Qwen3.5-122B-A10B (122B) ✓
M2 24 GB	Qwen2.5-7B 4-bit+QJL, small MoEs	comfortably larger MoEs
M3 48 GB	Nemotron-120B hybrid (Part 4)	100B+ at higher speed
M3 64 GB	GPT-OSS-120B, Qwen3.5-122B (Parts 2-3)	100B+ at full speed



Apple Silicon RAM tier vs largest runnable TurboQuant model. Expert streaming takes the 16 GB row from ~20B all the way to 122B — by decoupling model size from RAM. Source: Author's experiments. Source: Manjunath Janardhan experiments.

Quantization shrinks the model. Streaming **decouples model size from RAM entirely** — and that’s the move that puts a 122B on the 16 GB tier.

The two models

I used two MoEs from the Qwen family — same `qwen3\_5\_moe` architecture, which mattered later (one loader ran both).

Model Comparison		
Property	Qwen3.6-35B-A3B	Qwen3.5-122B-A10B
Total parameters	~35 B	~122 B
Active per token	~3 B	~10 B
Routed experts	256 + 1 shared	256 + 1 shared
Active experts / token	8	8
Layers	40	48
Attention	hybrid linear + softmax	hybrid GatedDeltaNet + softmax
BF16 size	~70 GB	~240 GB
TurboQuant 3-bit	~16 GB	~54 GB

The “A3B”/”A10B” is the important part: only ~3B and ~10B parameters are *active* per token. That sparsity is the whole reason streaming works — and 256 experts give 3-bit quantization plenty of redundancy to stay coherent (a finding from Part 2: high-expert-count MoEs absorb 3-bit noise; a 32-expert model would not).

I converted both with the standard data-free TurboQuant 3-bit recipe (Hadamard rotation + Lloyd-Max codebook). On a 64 GB M4 Max they run fully resident at ~60 and ~26.5 tok/s. But ~18 GB and ~55 GB peak is exactly what *doesn’t* fit a 16 GB mini.

The wall: quantization alone doesn’t fit 16 GB

You might hope MLX’s lazy loading saves you — memory-map the weights and let the OS page them in on demand. I hoped that too. **It doesn’t work**, and understanding why is the key to everything that follows.

When you `load(..., lazy=True)`, the weights are mmap-backed and unmaterIALIZED — RSS is tiny. But the *first forward pass* runs a matmul against every expert tensor, and MLX has to materialize each full `(256, ...)` stacked expert array into a Metal buffer to run the kernel. After one token, all 256 experts of every layer are resident, you're back at 16+ GB, and the machine swaps. Lazy loading defers the cost; it doesn't avoid it.

So the experts have to be loaded a **few at a time, explicitly, per token** — and freed afterwards. That's the work.

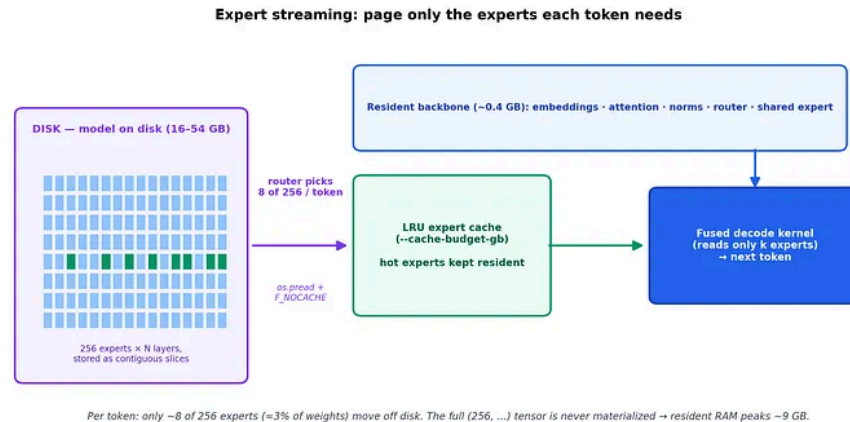
The idea: stream only the experts each token needs

The MoE router picks 8 of 256 experts per token. Those 8 weight blocks are the only expert data the forward pass touches. Everything else — embeddings, attention, norms, router, the shared expert — is small and stays resident. So:

1. Keep the **non-expert “backbone” resident** (a few hundred MB).
2. For each token, read the router's choice and **page in just those experts' weight slices from disk**.
3. Hold recently-used experts in a byte-budgeted **LRU cache** so a hot expert isn't re-read every token.
4. Never let the full `(256, ...)` expert tensor materialize.

This is cheap on disk because of the same sparsity that makes MoE inference cheap on compute. TurboQuant stores each expert projection as a stacked tensor laid out **contiguously along the expert axis**, so expert e is a single contiguous byte range — pulling it is one `pread` of a known offset and length. The decode kernels from Part 2 already read *only the k selected experts*, so streaming is mostly: assemble a small `(k, ...)` stack from the paged-in

experts, remap the routing indices to local positions, run the existing kernel. The math is untouched — which is why the output is **bit-identical** to the fully-resident model (verified: greedy decode produces an identical token stream; per-step max-abs-difference is exactly 0).



Expert streaming. The resident backbone stays in RAM; per token, the router selects 8 of 256 experts and only those contiguous slices are pread from disk into a byte-budgeted LRU cache, then fed to the existing fused decode kernels. The full (256, ...) tensor is never materialized. Source: Manjunath Janardhan experiments.

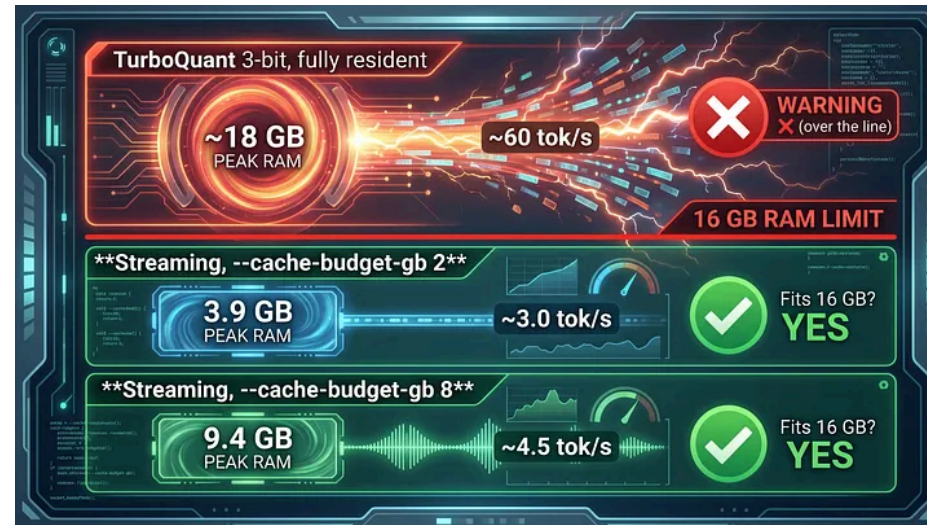
The trap that cost me a day: page-cache bloat

The first working version streamed correctly but RSS still ballooned to **12.5 GB** — almost as bad as not streaming. MLX’s *managed* memory was a healthy ~5 GB, so where was the other 7 GB? The OS page cache. Reading 14+ GB of expert slices through normal mmap I/O, macOS cached every page in the unified buffer cache — clean and evictable, but on a 16 GB machine 12.5 GB RSS is the difference between “runs” and “beachball.”

The fix is two macOS specifics: open each shard’s fd with ``F_NOCACHE`` (``fcntl(fd, F_NOCACHE, 1)``) so the OS doesn’t retain its pages, and read slices with ``os.pread`` instead of mmap-slicing. RSS dropped from 12.5 GB to ~ the MLX managed memory. ``F_NOCACHE`` is the single most important line in the streaming reader — without it, none of this fits 16 GB.

First proof: a 35B model in 3.9 GB

Before the 122B, the 35B. Measured on a **base Apple M4 Mac mini, 16 GB**, streaming the 3-bit Qwen3.6–35B-A3B:



A 35B model resident in 3.9 GB, with the backbone-only footprint at load just 0.45 GB. The cache budget is a dial: bigger cache → higher expert hit-rate → fewer disk reads → faster. At 2 GB the hit-rate was ~60% (~3 tok/s); at 8 GB it hit 91% and ~4.5 tok/s, still leaving ~6.5 GB free. The 35B was the proof that the technique works and stays coherent. Then the real question: how far up does it scale?

Pushing it: a 122B on the same machine

I pointed the exact same ``stream_generate`` at Hugging Face Repo ``manjunathshiva/qwen3.5-122b-tq3``. **Zero code changes** — Qwen3.5–122B is the same ``qwen3_5_moe`` family as the 35B (also multimodal, so the same tensor layout), so the loader's expert path matched as-is. (Group size 64 vs the 35B's 32 made no difference — the reader is shape-driven and the loader passes the model's real group size to the kernels.)

First run, a conservative `— cache-budget-gb 1`:

```
[stream] loaded in 138.6s | resident RSS=0.41 GB  
Generation: 64 tokens, 0.65 tok/s | Peak memory: 6.0 GB  
[stream] expert cache: hit_rate=0.0% disk_read=113.8 GB
```

It ran. A 122B model, generating a coherent Rayleigh-scattering explanation, on a 16 GB Mac. The backbone resident footprint was **0.41 GB**— even with a shared expert per layer, the non-streamed weights are tiny. Peak was **6.0 GB**. My worry that the 122B backbone wouldn't fit was completely unfounded.

Then I tried to go faster by raising the cache to 8 GB — and it **crashed**:

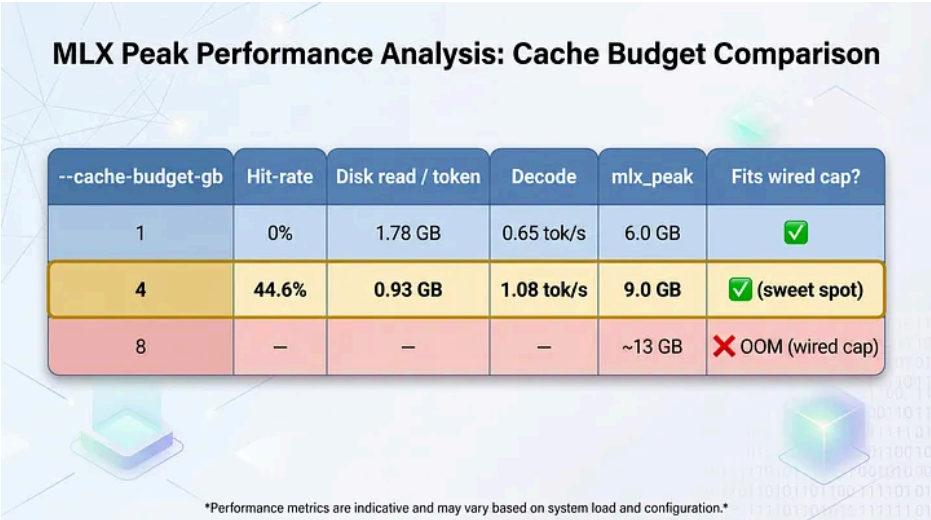
```
[METAL] Command buffer execution failed: Insufficient Memory
```

This is the most interesting finding in the whole project.

The real ceiling isn't RAM — it's the Metal wired-memory cap

The crash wasn't total RAM (the 122B had peaked at 6 GB). It was the **Metal GPU wired-memory cap**. On a 16 GB Mac, the GPU can only wire ~10.5 GB of unified memory, and the expert cache lives in *wired* MLX memory — so an 8 GB cache plus the ~5 GB base footprint tried to wire ~13 GB and blew past the cap.

That gives a clean rule of thumb: ``mlx_peak ≈ 5 GB base + cache_budget``. So on a 16 GB mini, the cache tops out around 4–5 GB. I re-ran at `— cache-budget-gb 4`:

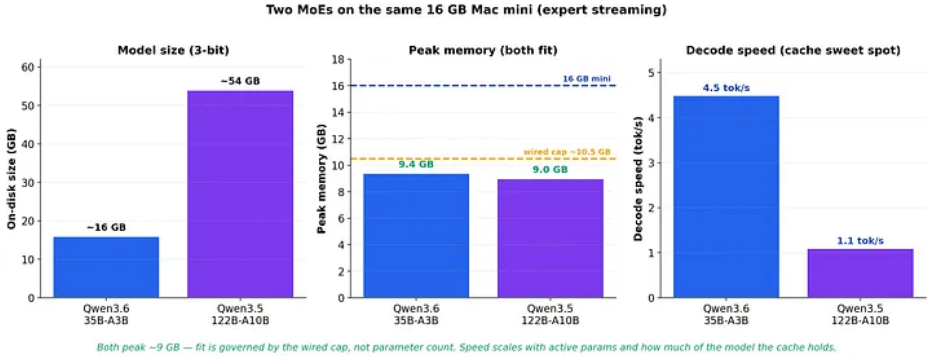
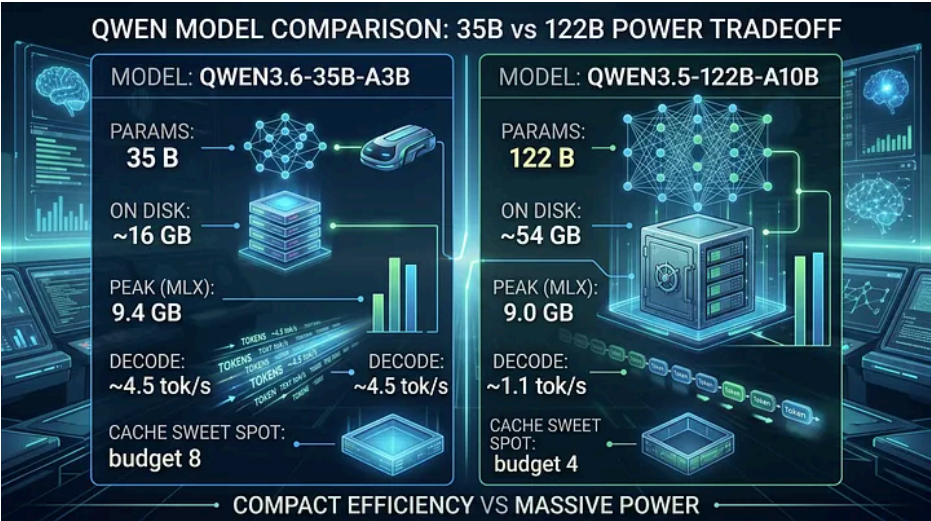


122B on a 16 GB mini. Raising the cache lifts the expert hit-rate (0% → 44.6%) and decode speed (0.65 → 1.08 tok/s), but the cache lives in wired GPU memory, so mlx_peak ≈ 5 GB + budget hits the ~10.5 GB Metal wired cap. Budget 4 is the sweet spot; budget 8 OOMs. Source: Manjunath Janardhan experiments.

Budget 4 is the sweet spot: the hit-rate jumped from 0% to 44.6% (the model’s “popular experts” get cached — routing locality is better than I expected), per-token disk reads halved, and decode crossed 1 tok/s, all at 9 GB peak with safe margin under the cap.

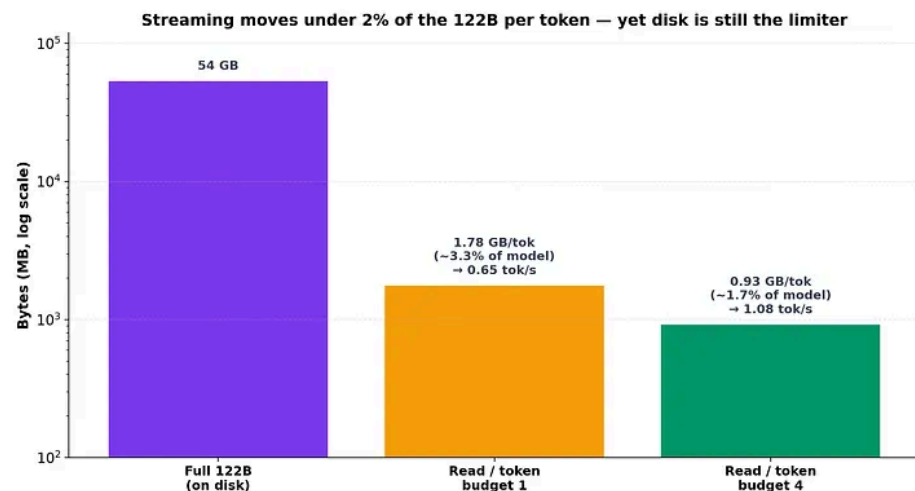
The headline number is ~1 token/sec. That’s slow — a 122B activates ~10B params per token and reads ~1 GB off the SSD each step, and the cache can only cover ~9% of a 54 GB model under the wired cap. **Disk bandwidth is the wall now, not memory.** But “slow” is the right kind of problem to have: a 122B model *runs at all* on a 16 GB Mac, and the limiter is cheap, plentiful disk rather than scarce, expensive RAM.

Two models, one 16 GB machine



Both models on the same 16 GB Mac mini. The 35B runs faster (smaller active set, cache covers half the model); the 122B is disk-bound at ~1 tok/s but fits just as comfortably in memory. Fit is trivial; speed scales with active params and how much of the model the cache can hold. Source: Manjunath Janardhan experiments.

The pattern: **fit is governed by the wired cap and the (tiny) backbone, not the parameter count.** Both models peak around 9 GB. Speed is governed by active params per token and what fraction of the model the cache can hold — which is why the 122B is ~4× slower than the 35B despite fitting just as easily.



Throughput tracks bytes-read-per-token, not model size. A bigger cache reads fewer bytes/token (the SSD is the wall), so decode speeds up.

Streaming moves a tiny fraction of the model per token. The 122B reads ~0.93 GB/token at budget 4 — under 2% of its 54 GB — yet that's still the throughput limiter. Decode speed tracks bytes-read-per-token, not model size. Source: Manjunath Janardhan experiments.

The caveats

Speed. ~1 tok/s for the 122B (and ~3–4.5 for the 35B) is a reading-pace experience, not real-time chat. Streaming is the right tool when the alternative is “the model doesn’t run at all.” If you have a 64 GB Mac, run it resident.

The wired cap caps the cache. On 16 GB you can’t grow the cache past ~4–5 GB without a Metal OOM. You *could* raise ``iogpu.wired_limit_mb`` with ``sysctl``, but on a 16 GB machine that starves everything else — and “no ``sysctl`` tweaks” is part of the point. The real fix (future work) is to hold cached experts in *regular* RAM and wire only the active 8 per token, decoupling cache size from the cap.

SSD reads. At budget 4 the 122B reads ~0.93 GB/token. Fine for interactive use, but not nothing — a bigger cache (where the cap allows) is gentler on the drive as well as faster.

Thinking-mode models. Both Qwen models emit a ``<think>`` trace before the final answer, so give them a generous ``— max-tokens` (512+)`. Neither needs a ``— min-tokens` floor`.

Architecture coverage. The loader currently targets the ``qwen3_5_moe` expert layout`. Generalizing to other MoEs (GPT-OSS, Nemotron's latent-MoE) is the obvious next step — the per-expert byte-slice primitive is general; only the layer-path wiring is model-specific.

Try it yourself

Everything here ships as one PyPI package and two HuggingFace repos. A 16 GB Mac is enough.

```
# 1. Install (pure Python; the Metal kernels JIT-compile at first use)
pip install "turboquant-mlx-full>=0.4.1"

# 2a. The 122B (~54 GB on disk; ~1 tok/s; budget 4 is the sweet spot on 16 GB)
python -m turboquant_mlx.stream.stream_generate \
  - model manjunathshiva/qwen3.5-122b-tq3 \
  - prompt "Explain why the sky is blue." \
  - max-tokens 128 - cache-budget-gb 4

# 2b. Or the faster 35B (~16 GB on disk; ~4.5 tok/s at budget 8)
python -m turboquant_mlx.stream.stream_generate \
  - model manjunathshiva/Qwen3.6-35B-A3B-tq3-g32 \
  - prompt "Explain why the sky is blue." \
  - max-tokens 512 - cache-budget-gb 8
```

Each run reports resident RSS, decode tok/s, expert hit-rate, and total disk read. Tune ``— cache-budget-gb` up for speed (until you near the Metal wired cap, ~10.5 GB on a 16 GB Mac) or down for a tighter RAM envelope`.

From Python, the loader swaps the expert layers for you:

```
from turboquant_mlx.stream.loader import load_streaming
from mlx_lm import generate
from mlx_lm.sample_utils import make_sampler
model, tokenizer, cache = load_streaming(
    "manjunathshiva/qwen3.5-122b-tq3", cache_budget_gb=4,
)

text = generate(model, tokenizer,

prompt="Why is the sky blue? Explain in detail.",

max_tokens=128, sampler=make_sampler(temp=0.7), verbose=True)

print(cache.stats()) # hit_rate, resident_gb, bytes_read_gb
```

Found something interesting on a config I haven't tested — a 32 GB machine, an external NVMe, a different cache budget? File an issue or PR on the <https://github.com/manjunathshiva/turboquant-mlx> . The 16 GB tier is wide open.

What I learned

Sparsity is a memory technique, not just a compute technique. MoEs were designed so you only *compute* a fraction of the network per token. The same property means you only need a fraction of the weights *in memory* per token — if you do the bookkeeping. Streaming takes the architecture's own promise literally.

Lazy loading is a trap on MLX for this. “Memory-mapped, loads on demand” sounds like it should stream for free. It doesn't — the first matmul materializes the full tensor. Bounding memory means loading experts explicitly and freeing them.

`F\_NOCACHE` is the unsung hero. The difference between 12.5 GB and 4 GB RSS was one `fcntl` flag plus switching mmap-slicing to `pread` . On a

memory-constrained machine, controlling the OS page cache matters as much as controlling your own allocations.

On a 16 GB Mac, RAM isn't the ceiling — the Metal wired-memory cap is.

Both a 35B and a 122B peak around 9 GB; the binding constraint is the ~10.5 GB GPU wired cap, and the cache lives in it. Knowing the *actual* limiter changed how I tuned everything.

The memory wall is a bandwidth wall in disguise. Once a model streams, “how much RAM” stops being the question and “how fast is my SSD, and how much can the cache hold” takes over. A 122B fits a 16 GB Mac trivially; it’s “slow” only because disk is the limiter — and disk is cheap. That reframing is the real result, bigger than any single model.

Resources

- TurboQuant paper <https://arxiv.org/abs/2504.19874> — Zandieh, Han, Daliri, Karbasi (2025).
- TurboQuant-MLX on PyPI <https://pypi.org/project/turboquant-mlx-full> —
`pip install "turboquant-mlx-full>=0.4.1"``
- Qwen3.5-122B-A10B-tq3 on HuggingFace
<https://huggingface.co/manjunathshiva/qwen3.5-122b-tq3> — the 122B streaming model card.
- Qwen3.6-35B-A3B-tq3-g32 on HuggingFace
<https://huggingface.co/manjunathshiva/Qwen3.6-35B-A3B-tq3-g32> — the 35B streaming model card.
- TurboQuant-MLX on GitHub
<https://github.com/manjunathshiva/turboquant-mlx> — Source, issues, Apache-2.0.

Support

If you found this article informative and valuable, I'd greatly appreciate your support:

“Give it a few claps 🙌 on Medium to help others discover this content (did you know you can clap up to 50 times?). Your claps will help spread the knowledge to more readers.”

- Share it with your network of AI enthusiasts and professionals.
- Subscribe to my YouTube channel for AI videos explained in simple English: <https://www.youtube.com/@AIBroEnglish>
- Connect with me on LinkedIn: <https://www.linkedin.com/in/manjunath-janardhan-54a5537/>

Thanks For Reading

Turbo Quant

Mlx

Qwen

Mac Mini



Published in Data Science Collective

937K followers · Last published 4 days ago

Advice, insights, and ideas from the Medium data science community

Follow



Written by Manjunath Janardhan

1.3K followers · 88 following

AI/ML Computational Science Senior Manager at Accenture with 21+ years building enterprise AI, GenAI, and intelligent operations.

Follow

Responses (24)

D

Deividas Ruseenas

What are your thoughts?

R

Reski Rukmantiyo

May 27

Just wondering about the quantity. Most of these techniques, will sacrifice model quality. How about this?

23

1 reply

[Reply](#)

Andrea Milan

May 27

Interesting but honestly meaningless if you do t talk about the quality of the answers...

18

1 reply

[Reply](#)

R. Thompson (PhD) he/him

May 27

This is a truly impressive technical achievement. Running a 122B-parameter MoE model on a 16GB Mac mini by leveraging per-expert disk streaming is a game-changer for accessible AI research. It really pushes the boundaries of what we thought was possible on consumer hardware. Great work on TurboQuant-MLX!

5

1 reply

[Reply](#)

See all responses

More from Manjunath Janardhan and Data Science Collective

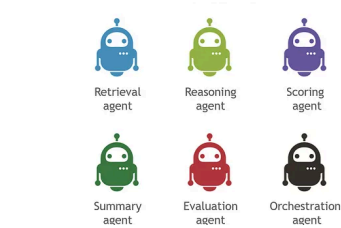


In Mac O'Clock by Manjunath Janardhan · Jul 3

Qwen3.6-35B Runs on a 16 GB M4 Mac mini — Fully in Memory, No...

Not a 2B toy model: a 35-billion-parameter AI now fits entirely inside the memory of the...

896 14 4

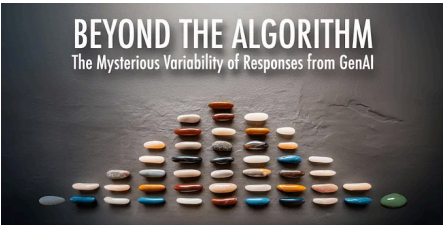


In Data Science Collective by Erdogan T · Jun 12

A Step-by-Step Guide for Developing Your Personal Agenti...

A complete guide to learn how to set up and create your own agentic LLM system with...

1.5K 28 33



In Data Science Collective by Hayanan · Jun 1

The Most Mysterious AI Response Ever Recorded: When Machines...

Six unexplained AI outputs that researchers, journalists, and the models' own creators stil...

3.5K 114 23



In Data Science Col... by Manjunath Janar... · Jun 18

Run Claude Code Locally on a Mac: 65 tok/s with a 4-bit Qwen3.6-27...

A step-by-step guide to driving Claude Code entirely offline on Apple Silicon — no cloud,...

542 11 8

See all from Manjunath Janardhan

See all from Data Science Collective

Recommended from Medium

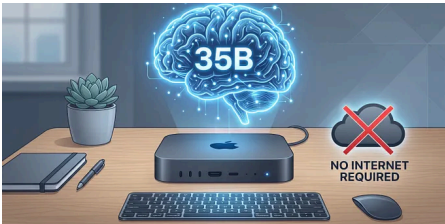


In No Time by Pranit naik · Jun 23

Japan Just Beat Claude Mythos And Nobody Saw It Coming

Sakana Fugu: Here Is Everything You Need To Know About This AI

979 18 23

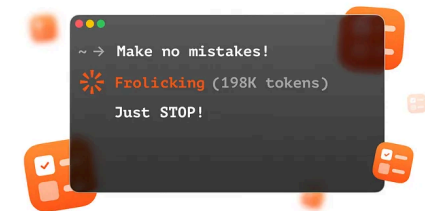


In Mac O'Clock by Manjunath Janardhan · Jul 3

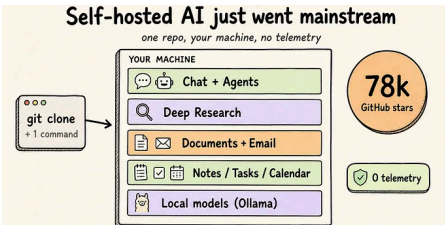
Qwen3.6–35B Runs on a 16 GB M4 Mac mini — Fully in Memory, No...

Not a 2B toy model: a 35-billion-parameter AI now fits entirely inside the memory of the...

896 14 4



Michal Malewicz · Jun 29

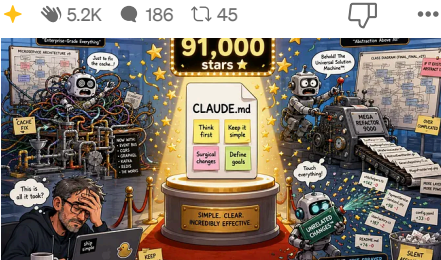


In Data Science Collective by Andrus · Jul 1

A YouTuber Just Did More for Self-Hosted AI Than a Decade of Open...

You only have weeks left to vibe code

Then it's over. You better hurry up!



In Towards Deep Learn... by Sumit Pand... · May 8

A Single CLAUDE.md File Went Viral. The Reason Is...

91,000 stars on GitHub. No code. Four rules from Andrej Karpathy that every coding...

5.2K 186 45

For about a year now I have been running a small, slightly embarrassing pile of software...

1.8K 23 15



In GoPenAI by Andrew Zhu · May 23

Why You Should Completely Avoid Ollama in 2026

And the way better open source options

2K 33 11

See more recommendations